# CLOUD DEVOPS ENGINEER

# Cybersecurity & Digital Banking Platform Interview Reviewer

Macquarie — Role-specific preparation guide

Linux • Cloud Operations • WAF • OWASP • Logging & Monitoring • Incident Response • Scripting • CI/CD • AI

## How to use this reviewer

Review the concepts, practise explaining your decisions out loud, and adapt the sample answers to your real experience. Do not claim hands-on experience with a tool or responsibility you have not used. For scenario questions, explain your investigation steps, risk controls, validation, communication, and lessons learned.

Prepared for interview study • October 2026

# 1. Understand the Role

This role maintains and improves cloud platforms that help protect applications from malicious activity. It combines DevOps operations with security engineering: reliable infrastructure, secure configuration, useful telemetry, incident investigation, automation, and controlled change.

## What the interviewer is likely assessing

- Cloud and Linux fundamentals: networking, identity, permissions, patching, service health, and troubleshooting.
- Web Application Firewall (WAF) understanding: managed rules, custom rules, false positives, logging, tuning, and safe rollout.
- Threat awareness: OWASP risks, suspicious traffic patterns, attack indicators, and escalation.
- Observability: log searches, dashboards, alerts, correlation, and turning signals into useful investigations.
- Incident response: prioritisation, containment, evidence preservation, recovery, communication, and post-incident learning.
- Automation and delivery: Python or shell scripting, CI/CD pipelines, testing, approvals, rollback, and secure secrets handling.
- Judgement in a banking context: customer impact, availability, data confidentiality, auditability, and change control.

## A strong answer pattern

**Context** → **Signal** → **Investigation** → **Action** → **Validation** → **Prevention.** Explain what alerted you, how you narrowed the scope, what evidence you used, how you reduced risk, how you confirmed recovery, and what you improved afterward.

# 2. Cloud and Linux Fundamentals

## Cloud concepts to revise

- **Identity and access:** least privilege, role-based access, short-lived credentials, MFA, service identities, and reviewing excessive permissions.
- **Network controls:** VPC/VNet, subnets, route tables, security groups/firewalls, network ACLs, private endpoints, DNS, TLS, and ingress/egress paths.
- **Resilience:** multiple availability zones where appropriate, health checks, autoscaling, backups, tested recovery, and clear recovery objectives.
- **Security posture:** patching, hardened images, encryption at rest/in transit, secrets management, configuration monitoring, and audit logs.
- **Operations:** change management, deployment validation, capacity monitoring, cost awareness, and documented runbooks.

## Linux commands worth knowing

| Purpose | Examples |
|---|---|
| System and resources | uname -a; uptime; free -h; df -h; du -sh /path |
| Processes and services | ps aux; top; systemctl status <service>; journalctl -u <service> |
| Network | ip addr; ip route; ss -tulpn; curl -v https://host; dig host; getent hosts host |
| Logs and text | tail -f /var/log/<file>; grep -i 'error' file; awk; sed; less |
| Permissions | ls -l; id; chmod; chown; sudo -l |

| Purpose | Examples |
| --- | --- |
| TLS/certificates | openssl s_client -connect host:443 -servername host |

### Q. A Linux-hosted service is returning 502 errors. What do you check first?

**Sample answer:** I would establish the scope and start time, then check the load balancer or proxy health, service status, recent deployments, application logs, CPU/memory/disk, and connectivity to dependencies. I would compare healthy and unhealthy instances, validate DNS and TLS if relevant, and use request IDs or timestamps to correlate logs. If a recent change is implicated, I would follow the rollback or mitigation process, verify recovery with health checks and error-rate metrics, and document the root cause.

### Q. How do you apply least privilege in cloud operations?

**Sample answer:** I grant only the permissions needed for a specific task, prefer roles and short-lived credentials over long-lived keys, separate human and workload identities, protect privileged actions with MFA and approvals, and periodically review access. I also log privileged activity and test that permissions work without granting broad administrator access.

# 3. Web Application Firewalls (WAF)

A WAF inspects HTTP/HTTPS requests and applies rules to detect or block malicious web traffic. It can help mitigate common web attacks, abusive requests, and known exploit patterns, but it is not a substitute for secure application code, identity controls, patching, or broader network security.

## Core concepts

- **Managed rules:** vendor-maintained signatures for common attack patterns; review rule versions and exclusions.
- **Custom rules:** conditions based on path, method, headers, IP reputation/allowlists, rate, geography, or other supported attributes.
- **Detection versus prevention:** monitor/count mode helps assess impact before blocking; prevention blocks matching requests.
- **False positive:** legitimate traffic is blocked. Tune narrowly, test against real application behaviour, and avoid disabling an entire protection set unnecessarily.
- **Rate limiting:** reduce abusive request volume, credential stuffing, scraping, or application-layer floods; set thresholds based on normal traffic.
- **WAF logs:** rule ID, action, client/source, URI, timestamp, request metadata, and request/correlation IDs when available. Treat sensitive fields carefully.

### Q. How would you deploy a new WAF rule safely?

**Sample answer:** I would define the threat and expected match conditions, review the impact on legitimate traffic, test in a lower environment, and use count or monitoring mode where supported. I would inspect matched requests, tune narrow exclusions if justified, obtain the required approval, then enable blocking gradually. I would monitor blocked/allowed rates, application errors, customer impact, and alert volume, with a documented rollback plan.

### Q. A WAF suddenly blocks many legitimate customers. What do you do?

**Sample answer:** I would confirm the scope and business impact, identify the specific rule and recent changes, and correlate WAF events with application logs and release timing. I would apply the approved mitigation—such as reverting the rule or adding a tightly scoped exclusion—rather than disabling the WAF globally. I would validate customer journeys, monitor for renewed attacks, communicate status, and follow up with a tested rule adjustment.

### Q. Does a WAF prevent SQL injection and cross-site scripting completely?

**Sample answer:** No. A WAF can detect and block many common patterns, but attackers can evade generic signatures and application context matters. The durable controls are parameterised queries, output encoding, input validation, secure frameworks, patching, and security testing, with the WAF acting as an additional defence layer.

# 4. OWASP and Web Threat Fundamentals

OWASP publishes widely used application-security guidance. Be ready to explain risks in practical terms and identify useful controls.

**Broken access control:** Users access data or actions they should not. Enforce server-side authorisation on every relevant request; test horizontal and vertical access.

**Cryptographic failures:** Sensitive data is exposed through weak or missing protection. Use modern TLS, appropriate encryption, key management, and data minimisation.

**Injection:** Untrusted input changes a query or command. Use parameterised queries, safe APIs, validation, and least-privileged database accounts.

**Security misconfiguration:** Unsafe defaults, excessive permissions, exposed admin interfaces, or verbose errors. Harden baselines and continuously review configuration.

**Authentication failures:** Weak login/session controls or credential attacks. Use MFA, rate limits, secure session handling, and risk-based detection.

**Vulnerable/outdated components:** Known vulnerable libraries or images. Maintain inventories, scan dependencies, patch by risk, and verify fixes.

**Logging and monitoring failures:** Events are missing, delayed, or not actionable. Centralise useful logs, protect them, alert on meaningful signals, and test response.

**SSRF:** A server is tricked into making unintended requests. Restrict outbound access, validate destinations, and protect cloud metadata services.

### Q. What OWASP risks are especially relevant to digital banking?

**Sample answer:** Broken access control, authentication/session weaknesses, injection, security misconfiguration, vulnerable dependencies, and insufficient logging are all important. I would connect each risk to banking controls: strong authorisation, MFA and fraud signals, secure coding, hardened cloud configuration, dependency scanning, and audit-quality monitoring. The exact priority depends on the application architecture and current threat model.

# 5. Logging, Monitoring, Searches, Rules, and Dashboards

## Key concepts

- **Logs** describe events; **metrics** quantify behaviour over time; **traces** show request flow across services.
- Centralise logs with consistent timestamps, time zones, service/environment labels, severity, correlation IDs, and structured fields.
- Build searches that narrow by time range, service, source, status code, rule ID, user/session identifier where lawful, and request ID.
- Dashboards should answer operational questions: Is the platform healthy? What changed? Which customers/services are affected? Is the attack continuing?
- Alerts should be actionable, have an owner and runbook, and be tuned to avoid noisy false positives and alert fatigue.
- Protect logs from unauthorised access or tampering; apply retention, redaction, and access controls appropriate to sensitive banking data.

### Example investigation searches (generic syntax)

- Find WAF blocks in a time window, grouped by rule ID and action.
- Compare 4xx/5xx rates before and after a deployment.
- Find repeated authentication failures by source, endpoint, and time bucket.
- Correlate a suspicious request ID across WAF, load balancer, application, and identity logs.
- Identify top endpoints by request volume and response latency; compare with baseline.

#### Q. What makes a good security dashboard?

**Sample answer:** It is designed around decisions, not just visual appeal. I would include WAF allow/block trends, top triggered rules, request rates, 4xx/5xx rates, latency, suspicious authentication activity, service health, and alert status. I would provide filters for environment, application, and time range, ensure timestamps and labels are consistent, and validate that metrics are trustworthy and actionable.

#### Q. How would you investigate a spike in blocked requests?

**Sample answer:** I would check whether the spike is isolated to one application, endpoint, source, or rule; compare it with baseline traffic and recent changes; inspect representative events; and correlate with application and network telemetry. I would assess whether it indicates scanning, exploitation, bot activity, or a false positive. I would escalate according to severity, tune rules only with evidence, and track whether the signal and customer impact return to normal.

## 6. Incident Response and Troubleshooting

### A practical incident workflow

- **1. Triage:** confirm alert validity, severity, affected service, customer impact, and whether the event is ongoing.
- **2. Preserve evidence:** record timestamps, query results, relevant log IDs, recent changes, and actions taken; avoid unnecessary exposure of sensitive data.
- **3. Scope:** identify affected accounts/services/endpoints, source patterns, related alerts, and potential lateral or downstream impact.
- **4. Contain:** use approved controls such as blocking a source, tightening a rule, isolating a workload, revoking credentials, or rolling back a change.
- **5. Recover:** restore known-good configuration, validate health and customer journeys, and monitor for recurrence.
- **6. Communicate:** provide concise updates with impact, actions, risks, owner, and next update time.
- **7. Learn:** conduct a blameless review, identify detection/control gaps, assign actions, and test improvements.

#### Q. You suspect a cyberattack against a banking application. What is your response?

**Sample answer:** I would treat it as a security incident, verify the signal and severity, and follow the incident-response runbook. I would correlate WAF, identity, application, and infrastructure logs to establish scope while preserving evidence. I would coordinate with the security incident lead and service owners, apply approved containment, and avoid changes that destroy evidence or create unnecessary customer impact. After recovery, I would monitor for recurrence and contribute to the post-incident review.

#### Q. How do you balance security and availability during an incident?

**Sample answer:** I assess the threat, confidence level, customer impact, and available mitigations. I prefer the narrowest effective control—for example, blocking a specific pattern or limiting a risky endpoint—rather than a broad shutdown, unless the risk justifies it. I communicate trade-offs, follow incident authority and change procedures, and verify both security effectiveness and service health.

### Q. What if an alert turns out to be a false positive?

**Sample answer:** I would document why it was false, confirm whether the detection logic or threshold needs tuning, and make a controlled change with testing and review. I would not simply suppress the alert without understanding the cause. I would also check whether similar alerts are noisy and ensure tuning does not create a blind spot.

# 7. Python and Shell Scripting

Use scripts for repeatable, auditable tasks—not for bypassing approvals or handling secrets unsafely.

## Useful scripting examples

- Parse JSON logs and summarise events by rule, status, or service.
- Check endpoint health and certificate expiry, with clear exit codes for CI/CD.
- Automate routine log collection or configuration validation.
- Call cloud APIs using managed identity/roles and least-privileged permissions.
- Validate required environment variables, input formats, and return codes.
- Add timeouts, retries with backoff, error handling, structured output, and useful logs.

### Q. How would you write a safe operational script?

**Sample answer:** I would define the inputs and failure conditions, validate arguments, use least-privileged credentials, avoid printing secrets, set timeouts, handle errors and retries carefully, and return meaningful exit codes. I would test normal and failure cases, keep it under version control, add logging without sensitive data, and include documentation and a rollback or recovery approach.

### Q. When would you choose Python over shell?

**Sample answer:** Shell is effective for short command orchestration and straightforward Linux tasks. Python is usually better for structured data, API integrations, reusable logic, testing, and more complex error handling. I choose the simplest maintainable tool for the job and avoid fragile parsing of command output when a stable API or structured format is available.

# 8. CI/CD and Secure Change Management

- A typical pipeline: source control → lint/static checks → unit/integration tests → security scans → package/artifact → approval where required → deploy → smoke tests → monitoring.
- Protect branches and pipeline identities; use reviewed changes, signed or traceable artifacts where applicable, and environment-specific permissions.
- Store secrets in a secrets manager or protected CI/CD secret store; never commit credentials or print them to logs.
- Use immutable artifacts and versioned infrastructure/application configuration so releases are reproducible.
- Use canary, blue/green, or staged rollout where appropriate; define health gates and rollback criteria.
- Infrastructure as Code should be reviewed, scanned, tested, and applied through controlled pipelines with state and credentials protected.

### Q. How do you prevent a risky change from reaching production?

**Sample answer:** I would use peer review, automated tests, security and configuration checks, environment separation, least-privileged pipeline identities, and approval gates based on risk. I would deploy a known artifact, verify health checks and key service metrics, and have clear rollback criteria. For urgent security changes, I would use the approved emergency-change path and complete documentation afterward.

### Q. How would you integrate security checks into CI/CD?

**Sample answer:** I would add appropriate static analysis, dependency and container scanning, secret detection, IaC checks, and tests at the stages where they give fast feedback. Findings should have severity thresholds, ownership, and a documented exception process. I would monitor scan failures and remediation time so the controls improve security without becoming ignored noise.

# 9. AI Tools in DevOps and Cybersecurity

AI can assist with summarising logs, drafting queries, explaining scripts, grouping alerts, and generating incident timelines. It should support—not replace—engineering judgement and established security processes.

### Q. How would you use AI tools in this role?

**Sample answer:** I would use approved AI tools to accelerate low-risk tasks such as explaining an unfamiliar log field, drafting a query, summarising a sanitized incident timeline, or generating a first draft of a script. I would verify the output against authoritative documentation and test it before use. I would never paste customer data, secrets, credentials, or restricted logs into an unapproved service, and I would not let AI autonomously make production security changes without the required controls.

## AI safety checklist

- Use only organisation-approved tools and data handling paths.
- Remove or mask sensitive information before submitting prompts.
- Validate generated commands, queries, code, and security recommendations.
- Treat model output as untrusted until reviewed and tested.
- Keep human approval, audit trails, and rollback for production-impacting actions.

# 10. Scenario Practice

### Q. Scenario 1: A new WAF rule blocks a payment endpoint after deployment. Walk through your actions.

**Sample answer:** Confirm customer impact and scope; identify the rule and deployment time; inspect representative blocked events; correlate with application errors; apply the narrowest approved rollback or exclusion; validate the payment flow with safe test transactions; monitor block/error rates; communicate status; and create a follow-up test to prevent recurrence.

### Q. Scenario 2: Authentication failures increase sharply from many IPs.

**Sample answer:** Check baseline and distribution, identify targeted accounts/endpoints and time patterns, correlate identity and WAF signals, and assess whether this resembles credential stuffing or another attack. Follow the security runbook, coordinate with the incident team, apply rate limits or other approved controls, protect account recovery flows, and monitor both attack reduction and legitimate-user impact.

### Q. Scenario 3: Logs are missing from a security dashboard.

**Sample answer:** Determine whether the issue is source generation, agent/forwarder, network, ingestion pipeline, parsing, indexing, permissions, or dashboard query. Compare source timestamps with ingestion timestamps, inspect pipeline health and recent changes, and verify with a known test event. Restore ingestion, assess the monitoring blind spot, alert the responsible team, and document any period where detection coverage was reduced.

### Q. Scenario 4: A deployment succeeds but latency and 5xx errors rise.

**Sample answer:** Compare before/after metrics, inspect release and infrastructure changes, check dependency health and resource saturation, and correlate request IDs with logs/traces. If rollback criteria are met, revert to the last known-good version. Validate recovery and then reproduce the issue safely to identify the cause.

### Q. Scenario 5: A critical vulnerability is reported in a production dependency.

**Sample answer:** Validate the affected versions and exposure, assess exploitability and business risk, check for available patches or mitigations, and coordinate with security and application owners. Use the approved emergency change process if needed, test compatibility, deploy the fix or mitigation, verify the running version, and retain evidence of remediation.

## 11. Sample Interview Questions — Quick Revision

### Q. What is the difference between a WAF and a network firewall?

**Sample answer:** A network firewall primarily controls network flows by addresses, ports, protocols, and related rules; a WAF focuses on HTTP/S application-layer requests and web attack patterns.

### Q. What is least privilege?

**Sample answer:** Giving each identity only the minimum permissions needed, for only as long as needed.

### Q. What is the difference between a metric and a log?

**Sample answer:** A metric is a numeric measurement over time; a log is an event record with context.

### Q. What is alert fatigue?

**Sample answer:** A high volume of low-value alerts makes important alerts easier to miss; tune thresholds and improve signal quality.

### Q. What is a false negative?

**Sample answer:** A real malicious event is not detected or blocked.

### Q. Why is TLS important?

**Sample answer:** It protects data in transit and helps authenticate the server when certificates are correctly validated.

### Q. Why centralise logs?

**Sample answer:** It enables correlation across systems, consistent retention and access control, faster investigations, and shared dashboards.

### Q. What is a rollback?

**Sample answer:** Returning to a known-good version or configuration when a release fails or causes unacceptable impact.

### Q. What is an incident postmortem?

**Sample answer:** A blameless review of causes, impact, response, and actionable prevention/detection improvements.

### Q. What should never be in logs?

**Sample answer:** Passwords, secret keys, access tokens, and unnecessary sensitive customer data; redact or mask fields.

### Q. What is rate limiting?

**Sample answer:** Restricting request volume over a time period to reduce abuse and protect service capacity.

### Q. Why is time synchronisation important?

**Sample answer:** Consistent timestamps allow reliable correlation of events across systems.

## 12. Questions You Can Ask the Interviewer

- Which cloud platforms and WAF technologies does the team currently operate?
- Which logging and monitoring platforms are most important for this role?
- How is responsibility shared between Cloud DevOps, application engineering, and the security operations/incident-response teams?

- What are the most common operational or security challenges the team is trying to improve?
- How are WAF rule changes tested, approved, and rolled back?
- What would success look like in the first 90 days?
- How does the team use AI tools safely in engineering and security workflows?

## 13. Suggested 30–60 Second Introduction

“I’m a DevOps Engineer with around six years of experience working across cloud infrastructure, automation, and CI/CD. My background includes AWS, Azure DevOps, Terraform, Docker, GitHub Enterprise, and security-related tooling such as SonarQube and GitHub Advanced Security. I’m interested in this role because it combines reliable cloud operations with security, monitoring, and continuous improvement. I approach incidents methodically: establish impact, use logs and metrics to narrow the cause, make controlled changes, validate recovery, and capture lessons learned. I’m keen to apply my DevOps experience while deepening my expertise in WAF operations and threat-focused monitoring.”

*Personalise this introduction to match your actual experience. If you have not directly managed a WAF or investigated a security incident, describe adjacent experience honestly and explain how you would approach the work.*

## 14. Final Preparation Checklist

- Prepare two real examples: one troubleshooting/incident example and one automation or CI/CD improvement.
- For each example, know the context, your specific actions, tools used, result, and what you learned.
- Review Linux process, service, disk, memory, networking, DNS, HTTP status, and TLS troubleshooting.
- Be able to explain WAF, OWASP risks, false positives, rate limiting, log correlation, and alert tuning.
- Practise explaining an incident without blaming others or making unsupported claims.
- Prepare to discuss secure secrets handling, least privilege, approvals, rollback, and customer impact.
- If you lack direct WAF experience, state that clearly, then connect related experience and show a sound learning plan.

Reference note: This reviewer is based on the supplied job description and general cloud/security engineering practices. Specific tools, rule names, and processes vary by organisation; follow Macquarie’s internal policies and approved runbooks.